

Full Combined Document

Table of Contents

v2 Overview

RM 简易步兵电控 deep-research v2

结论

这是在上一版学习包基础上，用 deep-research 技能 workflow 做过调研和迭代的新版本。
上一版不覆盖，v2 独立放在本文件夹。

文件结构

```
rm_infantry_control_deep_research_v2/  
  README.md  
  research_report.md  
  _skill_snapshot/  
    SKILL.md  
    INTRODUCTION.md  
    _meta.json  
  docs_docx/  
    00_v2_overview.docx  
    ...  
    10_full_combined_document.docx  
  docs_pdf/  
    00_v2_overview.pdf  
    ...  
    10_full_combined_document.pdf  
  improved_pack/  
    Core/  
    docs/  
    README.md  
    main_integration_example.c
```

推荐阅读顺序

1. research_report.md: 先看为什么要从 v1 迭代到 v2。

2. improved_pack/README.md: 看 v2 代码包结构。
3. improved_pack/docs/tutorial_step_by_step.md: 让提交者按步骤真正写懂。
4. improved_pack/docs/code_walkthrough.md: 逐函数讲清楚。
5. improved_pack/docs/test_plan.md: 按风险从低到高测试。

v2 新增能力

- 下载/快照了你指定的 .newmax deep-research 技能到 _skill_snapshot/。
- 新增调研报告 research_report.md。
- 新增 safety.c/h, 独立安全状态机。
- 新增 c620_can.c/h, 用于 RoboMaster 常见 C620/M3508 CAN 电调协议。
- 更新技术文档和教程, 加入官方资料校准后的解释。

边界

v2 仍然是应用层参考代码, 不是某块具体 STM32 板子的完整 CubeMX 工程。真实提交前, 需要在自己的板子上补 bsp_port.c 的 HAL 实现, 并把测试记录改成真实结果。

Source: E:/ai 产出文件/牛马/竞赛/竞

赛/outputs/robomaster/rm_infantry_control_deep_research_v2/
research_report.md:1

Deep Research and Iteration Report

RM 简易步兵电控考核 v2 深度调研与迭代报告

结论

v2 的目标不是把代码堆得更复杂, 而是把 v1 从“作业参考包”升级成“更接近 RoboMaster 电控工程习惯的学习包”。核心迭代有三点:

1. 新增 safety.c: 把急停、遥控丢失、Pitch 越界、输出限幅等安全状态集中管理。
2. 新增 c620_can.c: 补上 RoboMaster 常见 C620/M3508 CAN 电调协议打包和反馈解析。

3. 新增调研报告和答辩口径：让提交者能解释设计取舍，而不是只会说“代码能跑”。

调研范围

本轮调研聚焦“如何让 STM32 HAL 简易步兵电控作业更像真实 RM 电控工程”。

优先级：

- 官方硬件资料：RoboMaster C 型开发板、C620 电调手册。
- 官方代码/示例资料：STMicroelectronics STM32Cube HAL 示例、RoboMaster 开发板示例仓库。
- 已有作业 PDF：本地抽取的考核要求。

关键发现

1. C 型开发板决定了硬件适配层应该集中隔离

RoboMaster C 型开发板面向机器人控制，典型接口包含 STM32 主控、IMU、CAN、UART、PWM、DBUS 等。对考核作业来说，最重要的启发不是照搬某块板子的初始化代码，而是要把硬件接口隔离出来。

因此 v2 继续保留 `bsp_port.c`：

- 控制模块不直接调用 HAL。
- 换开发板时只改 BSP 层。
- 文档里明确说明哪些是真实硬件待接入。

2. C620/M3508 CAN 协议是 RM 电控常见能力，应作为加分理解点

虽然题目写的是普通直流霍尔编码器电机，但 RM 场景里常见的是 C620 电调配 M3508 电机。C620 手册给出了标准 CAN 控制和反馈格式，因此 v2 新增独立 `c620_can` 模块：

- `C620_PackCurrentFrame()`：把 4 路电流命令打包成 8 字节 CAN 帧。
- `C620_ParseFeedback()`：解析 0x201-0x208 反馈帧，得到角度、转速、转矩电流、温度。

- C620_LimitCurrent(): 把电流限制在合法范围。

这个模块不强制用于题目基本要求，但能在 README 或答辩中作为“我了解 RM 常见电机通信方式”的加分点。

3. 安全保护应该独立成状态机，而不是散落在 if 判断里

题目要求至少实现输出限幅、Pitch 限位、底盘急停中的两个。v1 已覆盖这些点，但逻辑分散在 app/chassis/gimbal 中。v2 把安全判断集中到 safety.c:

- Safety_Update() 统一更新安全状态。
- Safety_ShouldStop() 给调度层一个清晰决策。
- Safety_ReportMotorLimited() 让底盘/云台输出限幅可以被记录。

这样的好处是后续可以继续加入：

- 遥控器超时。
- 电机堵转。
- 电调过温。
- IMU 离线。
- 低电压保护。

4. 对考核最有价值的不是“全做完”，而是边界诚实

作业 PDF 明确写了“做多少算多少不要求全部做完”，并强调自主学习解决困难。提交时应该诚实区分：

- 已完成：软件架构、任务调度、运动学、PID 接口、安全保护。
- 已预留：编码器速度、IMU 角度、CAN 电机通信。
- 待实测：具体开发板引脚、电机驱动方向、PID 参数。

这比虚假承诺“已实车验证全部功能”更可信。

v1 到 v2 的具体迭代

代码迭代

- 新增 Core/Inc/safety.h
- 新增 Core/Src/safety.c
- 新增 Core/Inc/c620_can.h
- 新增 Core/Src/c620_can.c
- app.c 接入 Safety_Init()、Safety_Update()、Safety_ShouldStop()
- chassis.c 在输出限幅时上报安全状态
- README 更新 v2 新增模块

文档迭代

- 新增本报告：解释调研依据和迭代原因。
- 保留逐步教程：让提交者按模块真正写懂。
- 保留创作心得模板：但要求必须替换成真实调试经历。

提交者答辩口径

可以这样说：

我先按题目要求实现了模块化、底盘运动学、云台目标角和安全保护。后来我查了 RM 常见硬件，发现真实电控工程里 CAN 电调和集中安全状态很重要，所以 v2 里把 C620 CAN 帧打包和反馈解析单独做成模块，同时把安全判断从控制流程里拆成了 safety.c。

我没有把 HAL 代码写进底盘和云台模块，因为不同开发板的按键、PWM、CAN、IMU 接法不一样。把硬件适配放进 bsp_port.c 后，算法层就不会被具体引脚绑死。

这份代码目前是应用层框架和可移植参考，真实电机、编码器和 IMU 需要接入板子后继续验证。我会在测试记录里明确区分已软件验证和待实车验证的部分。

调研来源

- RoboMaster C 型开发板产品页：
<https://www.robomaster.com/en-US/products/components/general/development-board-type-c>
- RoboMaster C620 Brushless DC Motor Speed Controller 用户手册：<https://cdn-hz.robomaster.com/robomasters/public/document/RoboMaster%20C620%20Brushless%20DC%20Motor%20Speed%20Controller%20V1.0.pdf>
- STMicroelectronics STM32CubeF4 官方 HAL 示例仓库：
<https://github.com/STMicroelectronics/STM32CubeF4>
- RoboMaster Development Board C
Examples: <https://github.com/RoboMaster/Development-Board-C-Examples>
- 本地作业 PDF 抽取文本：
C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md

边界声明

本报告没有声称 v2 已经在真实机器人上跑通。当前已完成的是软件结构、协议打包/解析和本地 C 语法检查；真实硬件验证必须在 STM32Cube 工程中完成。

Learning Pack README

简易步兵机器人电控系统学习包

结论

这是一份面向 STM32 HAL 工程的 RoboMaster 简易步兵电控参考实现。它不是直接绑定某块板子的完整 CubeMX 工程，而是一个清晰的应用层框架：硬件相关内容集中在 bsp_port.c，控制逻辑按底盘、云台、遥控、PID、任务调度拆开。

最适合的使用方式：

1. 先读 docs/tutorial_step_by_step.md，理解每个模块为什么存在。
2. 把 Core/Inc 和 Core/Src 复制进自己的 STM32Cube 工程。
3. 按自己的板子改 bsp_port.c，接上按键、PWM/CAN、编码器、IMU。

4. 按 docs/test_plan.md 一项一项验证。
5. 最后改 README 和心得，把自己的调试过程写进去。

文件结构

```
Core/  
  Inc/  
    app.h  
    bsp_port.h  
    c620_can.h  
    chassis.h  
    gimbal.h  
    pid.h  
    remote.h  
    robot_def.h  
    safety.h  
  Src/  
    app.c  
    bsp_port.c  
    c620_can.c  
    chassis.c  
    gimbal.c  
    pid.c  
    remote.c  
    safety.c  
docs/  
  technical_report.md  
  design.md  
  code_walkthrough.md  
  tutorial_step_by_step.md  
  test_plan.md  
  reflection.md  
  submit_checklist.md  
main_integration_example.c
```

已覆盖功能

- 分层架构：app / remote / chassis / gimbal / pid / bsp_port
- 循环任务调度：App_Loop() 内部按周期调用控制任务
- 整车模式：ROBOT_STOP、ROBOT_NORMAL
- 底盘模式：独立模式、小陀螺模式、跟随云台模式
- X 型四轮全向底盘运动学
- 电机速度 PID 接口

- 云台 yaw/pitch 目标角控制
- Pitch 角度限位
- 急停、输出限幅、遥控超时保护接口
- 默认平移速度为 0，不会在解除急停后自动前进
- 按键短按、长按、长按连发
- 串口协议解析函数框架
- C620/M3508 CAN 电调协议打包与反馈解析模块
- 独立 safety 安全状态机
- L1 呼吸灯、L2 模式灯接口

需要自己接入的硬件

Core/Src/bsp_port.c 里所有函数都是硬件适配层：

- BSP_GetTickMs()
- BSP_ReadKey()
- BSP_SetMotorOutput()
- BSP_GetMotorSpeed()
- BSP_GetYawAngleDeg()
- BSP_GetPitchAngleDeg()
- BSP_SetLedL1()
- BSP_SetLedL2()

如果暂时没有实车，可以先用串口打印或调试变量观察目标速度、目标角度和输出值。

v2 调研迭代新增点

- 参考 RoboMaster 官方 C 型开发板资料，保留 IMU、CAN、UART、PWM、DBUS 等硬件接入位置。
- 参考 RoboMaster C620 官方用户手册，新增 0x200/0x1FF 电流控制帧打包和 0x201-0x208 反馈帧解析。
- 将安全逻辑从 app.c 中抽出为 safety.c，便于后续增加遥控丢失、堵转、过温、过流等保护。

- 文档新增 `research_report.md`，说明调研来源、设计取舍和 v1 到 v2 的迭代原因。

提交建议

提交时不要声称“所有硬件已经实测”，除非确实接过电机和 IMU。更稳妥的表达：

- 已完成软件架构、运动学、状态机、PID 接口、安全保护。
- 已预留编码器和 IMU 反馈接口。
- 当前硬件适配层需要根据实际开发板引脚和电机驱动方式配置。

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:11

Technical Report

简易步兵机器人电控系统技术文档

1. 项目目标

本项目基于 STM32 HAL 工程思想，实现一个简易步兵机器人电控系统的软件框架。系统覆盖底盘控制、云台控制、按键输入、PID 闭环、安全保护和任务调度，目标是形成一个可继续接入真实电机、编码器和 IMU 的工程基础。

2. 功能范围

已实现：

- 主控任务调度
- 遥控按键输入
- STOP/NORMAL 整车模式
- 独立、小陀螺、跟随云台三种底盘模式
- X 型四轮全向底盘运动学
- 四路底盘电机速度 PID
- yaw/pitch 云台目标角控制

- pitch 角度限位
- 云台角度 PID
- 电机输出限幅
- 急停保护
- L1 呼吸灯、L2 模式灯接口
- 串口数据包解析框架
- C620/M3508 CAN 电调协议打包和反馈解析
- 独立安全状态机

待接入/待实车验证：

- 具体 STM32 型号和引脚配置
- 真实 PWM/CAN 电机输出
- 编码器速度换算
- IMU yaw/pitch 角度读取
- 实车 PID 参数整定
- 具体 CAN 外设过滤器、发送邮箱和接收中断配置

3. 软件架构

App

```

├─ Remote: 按键和串口输入, 输出 remote_cmd_t
├─ Chassis: 底盘模式、运动学、轮速 PID
├─ Gimbal: 云台目标角、角度 PID、Pitch 限位
├─ PID: 通用闭环控制器
├─ Safety: 急停、遥控丢失、限幅等安全状态
├─ C620 CAN: RM 常见电调控制帧和反馈帧解析
└─ BSP Port: STM32 HAL 硬件适配层

```

架构设计原则：

- main.c 不放控制逻辑，只调用 App_Init() 和 App_Loop()。
- remote.c 只产生命令，不直接控制电机。
- chassis.c 只关心底盘速度和轮速输出。

- gimbal.c 只关心目标角、反馈角和云台输出。
- bsp_port.c 统一对接 GPIO、TIM、CAN、编码器、IMU。
- safety.c 只做安全状态汇总，不直接写电机。
- c620_can.c 只做协议打包和解析，不直接调用 HAL CAN。

4. 数据流

主控制周期为 $\text{ROBOT_CONTROL_PERIOD_MS} = 5 \text{ ms}$ 。

Remote_Update()

-> 生成 remote_cmd_t

Gimbal_Control()

-> 根据 key2/key3 更新目标角

-> 读取 IMU yaw/pitch

-> PID 输出云台电机控制量

Chassis_Control()

-> 根据底盘模式修正 vx/vy/wz

-> X 型全向轮解算

-> 读取编码器速度

-> PID 输出底盘电机控制量

5. 底盘控制

底盘输入为车体坐标系速度：

- vx：前后速度
- vy：左右速度
- wz：旋转角速度

X 型四轮全向轮速度分配：

$$w1 = +vx - vy - wz * K$$

$$w2 = +vx + vy + wz * K$$

$$w3 = -vx + vy - wz * K$$

$$w4 = -vx - vy + wz * K$$

其中 K 为底盘几何系数，实车时需要根据轮距和安装尺寸标定。

6. 云台控制

云台使用目标角闭环：

$\text{target_angle} - \text{imu_angle} \rightarrow \text{PID} \rightarrow \text{motor_output}$

Yaw：

- key2 短按增加目标角。
- key2 长按连续增加目标角。
- yaw 误差使用 Robot_WrapAngleDeg() 限制到 $[-180, 180]$ 。

Pitch：

- key3 短按增加目标角。
- key3 长按连续增加目标角。
- 目标角限制在 $[-20, 30]$ 。

7. 安全保护

安全保护优先级高于功能完整度：

- 急停：进入 ROBOT_STOP 后底盘和云台输出全部清零。
- 输出限幅：所有电机输出限制在安全范围内。
- Pitch 限位：从目标角源头限制，避免云台撞机械限位。
- 遥控超时接口：若后续接真实遥控器，应让 valid 和 last_update_ms 反映真实链路状态，并在超时后强制停机。

8. 硬件适配说明

所有硬件相关函数集中在 bsp_port.c：

```
uint32_t BSP_GetTickMs(void);
uint8_t BSP_ReadKey(key_id_t key);
void BSP_SetMotorOutput(motor_id_t motor, float output);
float BSP_GetMotorSpeed(motor_id_t motor);
float BSP_GetYawAngleDeg(void);
float BSP_GetPitchAngleDeg(void);
```

```
void BSP_SetLedL1(float duty_0_to_1);  
void BSP_SetLedL2(uint8_t on);
```

移植到真实工程时，只需要替换这些函数内部实现，不需要改控制层逻辑。

9. 完成度说明模板

当前版本完成了软件框架、状态机、运动学、PID 接口和安全保护。由于不同开发板引脚、电机驱动和 IMU 型号不同，硬件适配层需要根据实际设备补充。若已经接入实车，应在测试记录中补充电机悬空测试、编码器反馈测试和云台角度响应测试。

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:15

Design Notes

技术设计思路

结论

这个方案的核心不是“把车写动”，而是先把电控系统拆成可维护的控制链路：输入层负责产生命令，任务调度层负责节奏，底盘和云台负责闭环控制，BSP 层负责对接真实硬件。

总体架构

按键/串口/遥控器

|

v

remote.c 生成 remote_cmd_t

|

v

app.c 定时调度 ControlTask

|

+--> gimbal.c: 目标角 -> PID -> 云台电机输出

|

+--> chassis.c: vx/vy/wz -> 轮速 -> PID -> 底盘电机输出

|

v

bsp_port.c 连接 STM32 HAL、GPIO、TIM、CAN、编码器、IMU

为什么不能把逻辑堆在 main.c

RoboMaster 电控项目后期一定会多人协作：有人调底盘，有人调云台，有人接遥控器，有人调通信。如果所有逻辑都写在 main.c，任何人改一个功能都可能影响其他模块。模块化以后，每个文件只负责自己的对象，调试范围更小，后续接真实硬件也更稳。

状态机设计

整车状态：

- ROBOT_STOP：所有电机输出清零，PID 复位。
- ROBOT_NORMAL：底盘和云台正常工作。

底盘状态：

- CHASSIS_INDEPENDENT：底盘按自身坐标系运动。
- CHASSIS_SPIN：小陀螺，底盘加入固定旋转角速度。
- CHASSIS_FOLLOW_GIMBAL：用 yaw 偏差产生底盘旋转角速度，让底盘朝向追云台。

控制链路

底盘：

$v_x/v_y/w_z$ -> X 型全向轮解算 -> 4 个目标轮速 -> 4 路 PID -> 电机输出

云台：

key2/key3 -> 目标角变化 -> IMU 角度反馈 -> PID -> yaw/pitch 输出

安全保护

优先做三个保护：

- 输出限幅：所有电机命令经过限幅，避免疯转。
- Pitch 限位：目标角限制在 $[-20, 30]$ ，避免打机械限位。

- 急停：按键触发 `ROBOT_STOP`，所有输出立即清零。

和题目要求的对应关系

- 文件结构：chassis/gimbal/remote/pid/main
- 底盘：X 型全向轮运动学、编码器速度闭环接口
- 云台：yaw/pitch 目标角、pitch 限位、IMU 反馈接口
- 安全：输出限幅、pitch 限位、急停
- 进阶：任务调度、模式切换、串口协议解析框架

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:32

Code Walkthrough

代码逐函数讲解

结论

这份代码可以按“公共定义 -> PID -> 输入 -> 底盘 -> 云台 -> 调度 -> 硬件适配”的顺序理解。每个模块都遵守一个原则：只处理自己的职责，不跨模块乱改状态。

`robot_def.h`

作用：定义所有模块共享的类型和参数。

重点：

- `robot_mode_t`：整车 STOP/NORMAL。
- `chassis_mode_t`：独立、小陀螺、跟随云台。
- `remote_cmd_t`：遥控输入生成的统一命令。
- `Robot_LimitFloat()`：所有限幅都用它，避免输出超范围。
- `Robot_WrapAngleDeg()`：把 yaw 角误差限制到 $[-180, 180]$ 。

为什么这样写：公共定义集中后，后续调参数只改一个地方，不会在多个 `.c` 文件里找魔法数字。

pid.c

PID_Init()

设置 PID 参数、输出限幅、积分限幅，并清零内部状态。

PID_Reset()

清空积分和上次误差。急停时必须调用，否则恢复运行后可能因为积分残留导致电机突然冲一下。

PID_Calc()

计算逻辑：

```
error = target - feedback
integral += error * dt
derivative = (error - prev_error) / dt
output = kp * error + ki * integral + kd * derivative
```

为什么要积分限幅：防止长时间误差导致积分越积越大，恢复时输出失控。

remote.c

Remote_Init()

初始化按键状态和默认命令。默认进入 ROBOT_STOP，原因是上电默认停机更安全。

Button_Update()

内部函数，负责按键消抖、短按、长按、长按连发。

关键判断：

- 原始电平变化后先等待 REMOTE_DEBOUNCE_MS。
- 松开时如果没超过长按时间，判定为短按。
- 按住超过长按时间后，每隔 REMOTE_REPEAT_MS 产生一次连发事件。

Remote_Update()

把按键事件翻译成 remote_cmd_t：

- stop 短按切换 STOP/NORMAL。
- key1 短按切换小陀螺。
- key1 长按进入跟随云台。

- key2/key3 控制 yaw/pitch 目标角步进。

`Remote_ParseUartFrame()`

串口协议解析框架。它只负责把数据包解析成命令，不直接控制电机。

`chassis.c`

`Chassis_Init()`

初始化底盘状态和四个轮子的速度 PID。

`Chassis_CalcWheelTarget()`

把 $v_x/v_y/w_z$ 转成四个轮子的目标速度。这就是底盘运动学核心。

`Chassis_Control()`

控制流程：

1. 如果 STOP，调用 `Chassis_Stop()`。
2. 根据底盘模式修正 w_z 。
3. 计算四个目标轮速。
4. 读取四个编码器反馈。
5. 四路 PID 输出。
6. 输出限幅后发给电机。

`Chassis_Stop()`

目标速度、轮速输出、电机输出全部清零，并复位 PID。

`gimbal.c`

`Gimbal_Init()`

初始化 yaw/pitch 目标角和 PID 参数。

`Gimbal_Control()`

控制流程：

1. 如果 STOP，调用 `Gimbal_Stop()`。
2. 根据 key2/key3 更新目标角。
3. 对 pitch 目标角限位。

4. 读取 IMU yaw/pitch 角度。
5. 计算角度误差。
6. PID 输出云台电机控制量。
7. 输出限幅后发送给电机。

`Gimbal_Stop()`

清零云台输出并复位 PID。

`app.c`

`App_Init()`

初始化所有模块。

`App_Loop()`

主循环函数。它不直接写控制逻辑，只按时间调用 `App_ControlTask()`，同时更新 LED。

`App_ControlTask()`

控制任务核心：

```
Remote_Update(now_ms);
Gimbal_Control(cmd, dt_s);
Chassis_Control(cmd, gimbal->yaw_feedback_deg, dt_s);
```

为什么云台在底盘前面：底盘跟随云台模式需要用到最新 yaw 角反馈。

`bsp_port.c`

作用：把应用层和具体硬件隔离。

如果换开发板，只改这里：

- GPIO 按键
- LED
- PWM/CAN 电机输出
- 编码器反馈
- IMU 角度

为什么要隔离：控制算法不应该关心某个按键在哪个引脚，也不应该关心电机是 PWM 还是 CAN。

Source: E:/ai 产出文件/牛马/竞赛/竞

赛/outputs/robomaster/rm_infantry_control_learning_pack/Core/Src/app.c:49

Step by Step Tutorial

一步步真正写懂这份代码

结论

不要一上来复制全部文件。正确路线是每次只写一个模块，写完就问自己三个问题：输入是什么、输出是什么、出错时会怎样停下来。

第 0 步：先建 CubeMX 工程

目标：

- 能下载一个空工程到 STM32。
- 能让一个 LED 闪烁。

你需要确认：

- 使用的芯片型号。
- 按键 GPIO。
- LED GPIO 或 PWM。
- 电机驱动方式：PWM 还是 CAN。
- 编码器读取方式：TIM Encoder 还是外部中断。
- IMU 数据来源：SPI/I2C/UART/CAN。

没有这些信息也能先学软件结构，但不能说已经实车验证。

第 1 步：先写 pid.c

先理解 PID 的输入输出：

- 输入：目标值 target、反馈值 feedback、周期 dt_s
- 输出：控制量 output
- 核心误差：error = target - feedback

你要手写并测试：

```
PID_Init(&pid, 1.0f, 0.0f, 0.0f, -1000, 1000, -100, 100);  
out = PID_Calc(&pid, 100, 80, 0.005f);
```

如果 $k_p=1$ ，误差是 20，输出应接近 20。这个模块懂了，后面电机速度和云台角度都是同一种思想。

第 2 步：写 remote.c

先不要接复杂遥控器，只做按键：

- key1：切换底盘模式
- key2：yaw 目标角增加
- key3：pitch 目标角增加
- key_stop：急停/解除急停

关键不是按键本身，而是“消抖”和“短按/长按”。机械按键会抖动，如果不消抖，按一次可能被识别成多次。

你要能讲清楚：

- 为什么需要 REMOTE_DEBOUNCE_MS
- 为什么长按需要 REMOTE_LONG_PRESS_MS
- 为什么连发需要 REMOTE_REPEAT_MS

第 3 步：写 robot_def.h

这个文件只放公共定义：

- 整车模式
- 底盘模式
- 电机编号
- 限幅函数
- 关键参数宏

目的：让所有模块用同一套名字，避免到处出现魔法数字。

第 4 步：写 chassis.c

先只做运动学，不接电机：

```
w1 = +vx - vy - wz * K
w2 = +vx + vy + wz * K
w3 = -vx + vy - wz * K
w4 = -vx - vy + wz * K
```

你要用调试器观察：

- $v_x > 0$ 时四个轮子的目标值是否成对符合预期。
- $w_z > 0$ 时四个轮子是否形成旋转趋势。
- 输出是否被限幅。

然后再接 PID：

目标轮速 - 编码器反馈轮速 -> PID -> 电机输出

没有编码器时，BSP_GetMotorSpeed() 先返回 0，只能验证软件链路，不能说闭环实测完成。

第 5 步：写 gimbal.c

云台不要用编码器角度冒充 IMU。题目明确要求陀螺仪角度反馈。

先做目标角：

- key2 让 yaw 目标角每次加 5 度。
- key3 让 pitch 目标角每次加 3 度。

- pitch 目标角必须限制在 -20 到 30 度。

再做闭环：

目标角 - IMU 角度 -> PID -> 电机输出

如果没有 IMU，保留 BSP_GetYawAngleDeg() 和 BSP_GetPitchAngleDeg() 接口，并在 README 说明是待硬件接入。

第 6 步：写 app.c

app.c 是整个系统的节拍器：

```
Remote_Update();  
Gimbal_Control();  
Chassis_Control();
```

为什么先遥控，再云台，再底盘：

- 先读取输入，后续模块才有新命令。
- 云台先更新角度反馈，底盘跟随模式才能使用 yaw 信息。
- 底盘最后输出电机命令。

第 7 步：写 bsp_port.c

这里才接真实硬件：

- 按键：HAL_GPIO_ReadPin
- LED：HAL_GPIO_WritePin 或 PWM
- 电机：PWM 占空比或 CAN 电流
- 编码器：TIM Encoder 计数差分
- IMU：返回 yaw/pitch 角度

原则：硬件代码只放这里，不要散落到 chassis.c 和 gimbal.c。

第 8 步：理解 safety.c

安全模块不是为了多写一个文件，而是为了避免安全逻辑散落。

你要能讲清楚：

- `Safety_Update()` 每个周期收集当前安全状态。
- `Safety_ShouldStop()` 给调度层一个统一停机判断。
- `Safety_ReportMotorLimited()` 记录输出是否触碰限幅。

后续如果加入堵转、过温、低电压，也应该先进入 `safety.c`，再由 `app.c` 决定停机。

第 9 步：理解 `c620_can.c`

如果使用 RM 常见 C620 电调，需要知道两类 CAN 帧：

- 控制帧：主控发 4 路电流命令。
- 反馈帧：电调回传角度、转速、转矩电流、温度。

这个模块只负责“打包/解析”，不负责真正发送 CAN。真正的 `HAL_CAN_AddTxMessage()` 应该写在 `bsp_port.c` 或 CAN BSP 文件里。

第 10 步：逐项验证

不要直接上电让电机转。推荐顺序：

1. 只看 LED。
2. 只看按键状态。
3. 只看目标速度和目标角，不接电机。
4. 电机悬空，低限幅测试。
5. 接编码器反馈，看 PID 输出是否收敛。
6. 最后落地测试。

第 11 步：变成自己的代码

真正变成自己写的，不是改变量名，而是做到这三件事：

- 能画出模块关系图。
- 能说清楚每个函数的输入和输出。

- 能解释一次 bug 是怎么定位和修掉的。

建议他在 README 里写自己的调试记录，不要照抄模板。

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:90

Test Plan

测试计划

结论

测试顺序必须从低风险到高风险：先验证变量和 LED，再验证电机悬空，最后才落地跑车。

静态检查

- main.c 里只保留初始化和 App_Loop()。
- chassis.c 不直接调用 GPIO。
- gimbal.c 不直接调用 GPIO。
- remote.c 不直接控制电机。
- 所有硬件访问集中在 bsp_port.c。

功能检查

1. L1 呼吸灯
 - 现象：上电后亮度周期变化。
 - 失败排查：TIM PWM、GPIO 复用、BSP_SetLedL1()。
2. L2 模式灯
 - 现象：独立模式约 1Hz，小陀螺约 2Hz。
 - 失败排查：cmd->chassis_mode 是否切换。
3. 急停
 - 现象：按 stop 后所有电机输出为 0。

- 失败排查：ROBOT_STOP 是否传到 Chassis_Control() 和 Gimbal_Control()。
- 4. 底盘运动学
 - 现象：修改 vx/vy/wz 后四个 wheel_target 按公式变化。
 - 失败排查：轮子编号是否和实际安装一致。
- 5. 电机 PID
 - 现象：目标速度大于反馈速度时输出为正，接近目标后输出变小。
 - 失败排查：编码器方向、PID 参数、输出限幅。
- 6. 云台 Pitch 限位
 - 现象：反复按 key3，目标角不超过 30 度。
 - 失败排查：是否对目标角限幅，而不是只对输出限幅。
- 7. 云台 PID
 - 现象：目标角变化后，输出方向推动云台靠近目标。
 - 失败排查：IMU 角度方向、yaw 角 wrap、pitch 电机方向。
- 8. 遥控超时
 - 现象：接入真实遥控器后，通信中断会进入停机保护。
 - 失败排查：valid 和 last_update_ms 是否只在收到真实遥控数据时更新。

README 里建议记录的测试结果

- 测试日期
- 测试硬件
- 已验证功能
- 未验证原因
- 下一步计划

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:66

Reflection Template

创作心得模板

结论

这次作业最重要的收获不是写出某个函数，而是第一次按 RoboMaster 电控系统的思路，把“输入、状态、控制、反馈、安全”连成一条完整链路。

可以这样写

一开始我以为这个题目就是写几个电机控制函数，但拆开要求后发现，真正难的是系统结构。步兵机器人不是单独一个底盘，也不是单独一个云台，它需要在移动时保持云台稳定，还要能在异常情况下及时停机。所以我先没有直接写 `main.c` 里的大循环，而是把程序拆成了 `remote`、`chassis`、`gimbal`、`pid`、`app` 和 `bsp_port`。

我对底盘部分的理解是：遥控输入给的是车体速度 $v_x/v_y/w_z$ ，电机真正需要的是四个轮子的目标速度，所以中间必须有一层运动学解算。X 型全向轮的价值就在于可以把平移和旋转组合起来，但它也要求轮子方向和编号一致，否则车的实际运动会和预期相反。

我对云台部分的理解是：云台控制不能只看电机有没有转，而要看角度有没有跟上目标。题目要求使用陀螺仪反馈而不是电机编码器，这是因为云台最终要稳定的是姿态角，不是电机轴转了多少。Pitch 角度限位也是必须做的，因为机械限位被撞到后，轻则 PID 震荡，重则损坏机构。

我认为安全保护是这类机器人电控的底线。代码里我优先做了输出限幅、急停和 Pitch 限位。因为比赛现场最怕的不是功能少，而是机器人失控。即使后续继续扩展发射机构、裁判系统、视觉通信，也应该先保证任何状态下都有办法停下来。

这份代码还有很多可以继续完善的地方，比如真实编码器速度换算、IMU 驱动、串口协议校验、小陀螺参数整定和底盘跟随云台的实车调参。但通过这次作业，我已经理解了一个 RM 电控工程最基本的组织方式：先定义模块边界，再确定数据流，最后逐项接入硬件并测试。

必须改成自己的内容

提交前要把下面这些内容替换成自己的真实情况：

- 使用的 STM32 型号。
- 使用的开发板或自制板。
- 哪些功能真实上板测试过。
- 哪些功能只是软件接口完成。
- 遇到过什么 bug。
- 自己怎么定位问题。

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:92

Submit Checklist

提交检查清单

结论

提交前要让评审一眼看出三件事：结构清楚、功能有边界、作者真的理解。

必交内容

- STM32 工程源码
- README.md
- docs/design.md
- docs/test_plan.md
- docs/reflection.md
- Git 仓库链接
- 压缩包兜底

README 必写

- 项目目标
- 功能完成度

- 文件结构
- 按键说明
- 安全保护说明
- 硬件接入说明
- 未完成/待实车验证项

Git 提交建议

不要一次性提交全部代码。建议提交记录：

```
init stm32 hal project
add pid module
add remote key state machine
add chassis kinematics and speed pid
add gimbal angle control and pitch limit
add robot mode and safety stop
add docs and test record
```

压缩包命名

按题目要求命名为：

25+学院简称+姓名.zip

例如：

25 电科张三.zip

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:86